IF 260
Sistem Operasi

Minggu 5

Dr. Ananda Kusuma
e-mail: ananda.kusuma@lecturer.umn.ac.id

Universitas Multimedia Nusantara
Serpong, Tangerang

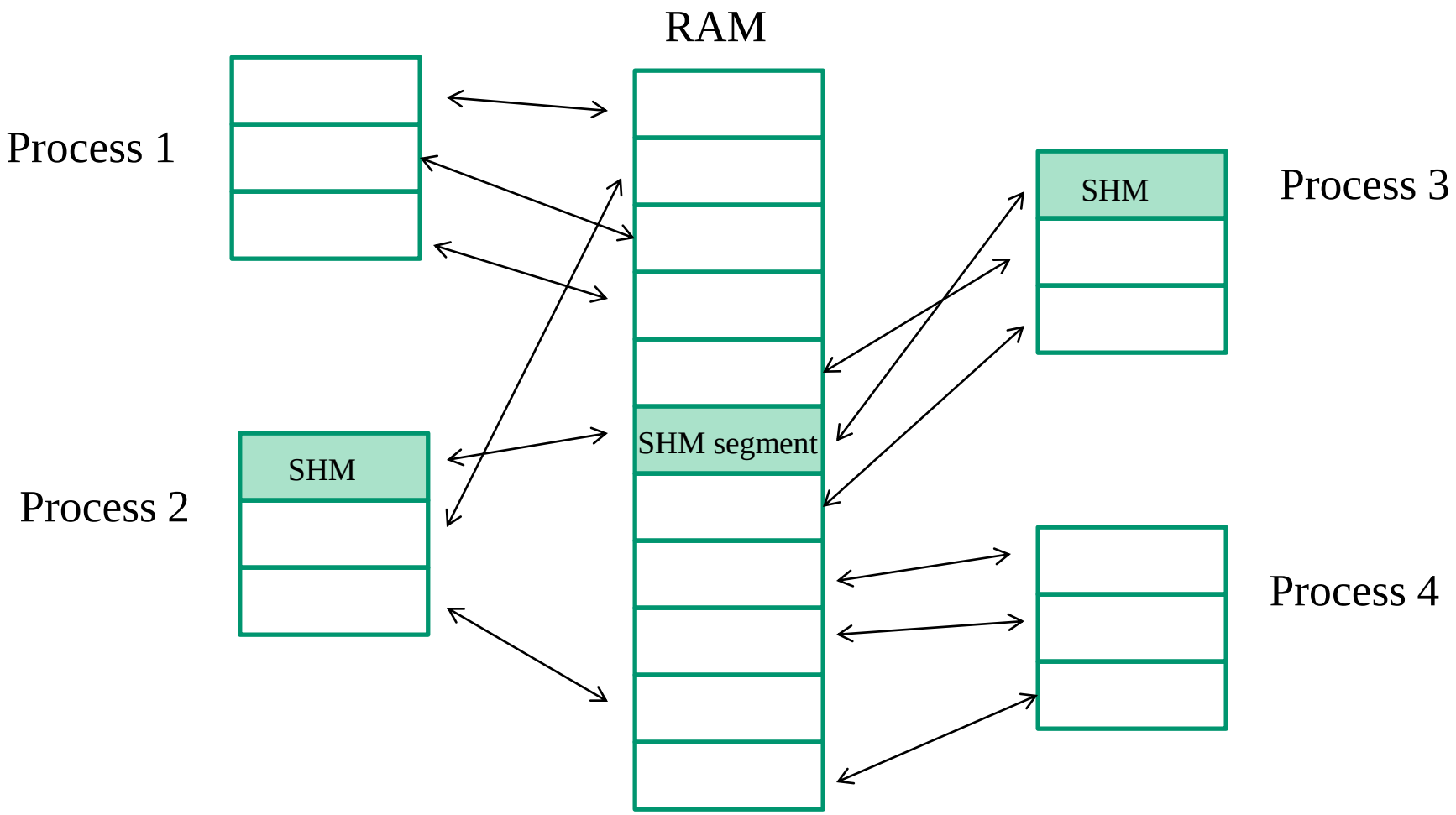
Agenda

- **Review**

- **Topik Minggu 5:**
  - **Mekanisme sinkronisasi**
    - Hardware
    - Software (synchronization primitives):
      - Semaphores,
      - Monitor

Review:
- Interprocess Communications (IPC)
(Komunikasi antar-Process)
- Race Condition

Contoh IPC:
Shared memory



Contoh IPC:
Shared memory

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE 1024  /* 1K byte shared memori */
int main(int argc, char *argv[])
{
   key_t  key;
   int shmid, mode;
   char *data;

   if(argc > 2){
      fprintf(stderr,"usage: ./demo-shm <data string>\n"); exit(1);
   }
   //membuat key
   key = ftok("shmdemo.c",'R');

   //connect ke segment memory atau create segment memory
   shmid=shmget(key,SHM_SIZE,0644 | IPC_CREAT);

   //attach to the segement to get a pointer to it
   data = (char *)shmat(shmid,(void *)0,0);

   //read or modify the segment, based on the command line
   if(argc == 2){
      printf("write: \"%s\"\n",argv[1]);
      strncpy(data,argv[1],SHM_SIZE);
   }
   else printf("read : \"%s\"\n", data);
//detach from the segment
   shmdt(data);
   return 0;
}
```

Contoh IPC:
Shared memory

```
ananda@AK-Ubuntu:~/COURSE-OS-OLD/VBOX-HOME/SHM$ gcc -o demo-shm demo-shm.c
ananda@AK-Ubuntu:~/COURSE-OS-OLD/VBOX-HOME/SHM$ ./demo-shm
read : ""
ananda@AK-Ubuntu:~/COURSE-OS-OLD/VBOX-HOME/SHM$ ./demo-shm "testing shm"
write: "testing shm"
ananda@AK-Ubuntu:~/COURSE-OS-OLD/VBOX-HOME/SHM$ ./demo-shm
read : "testing shm"
ananda@AK-Ubuntu:~/COURSE-OS-OLD/VBOX-HOME/SHM$ ./demo-shm "testing shm 2"
write: "testing shm 2"
ananda@AK-Ubuntu:~/COURSE-OS-OLD/VBOX-HOME/SHM$ ./demo-shm
read : "testing shm 2"
```

```
------ Shared Memory Segments --------
key        shmid      owner      perms      bytes      nattch     status
0x00000000 393216     ananda     600        524288     2          dest
0x00000000 1114113    ananda     600        16777216   2
0x00000000 1212418    ananda     600        524288     2          dest
0x00000000 1310723    ananda     600        524288     2          dest
0x00000000 819204     ananda     600        524288     2          dest
0x00000000 2129925    ananda     600        2097152    2          dest
0x00000000 1409030    ananda     600        524288     2          dest
0x00000000 1507335    ananda     600        524288     2          dest
0x00000000 2097160    ananda     600        524288     2          dest
0x00000000 1769481    ananda     600        524288     2          dest
0x00000000 1671178    ananda     600        33554432   2          dest
0x00000000 2490379    ananda     600        524288     2          dest
0x00000000 1966092    ananda     600        524288     2          dest
0x00000000 2228237    ananda     600        524288     2          dest
0x00000000 2719758    ananda     600        524288     2          dest
0x00000000 2293775    ananda     600        1048576    2          dest
0xffffffff 2752528    ananda     644        1024       0
```

ipcs -m

Producer-Consumer (Bounded-Buffer) Problem

- **Salah satu contoh classic sinkronisasi antar dua jenis processes atau threads:**
  - **Producer: membuat data dan menyimpan di buffer**
  - **Consumer: mengambil data dari buffer**
- **Terdapat shared buffer dengan kapasitas yang terbatas**
  - **Producer tidak dapat menyimpan tambahan data jika buffer sudah penuh → producer *sleep***
  - **Consumer tidak dapat mengambil data jika buffer kosong → consumer *sleep***
  - **Producer di-wakeup oleh consumer jika ada ruang kosong di buffer**
  - **Consumer di-wakeup oleh producer apabila ada data di buffer**
- **Apabila kedua process tidak sinkron (race condition), dapat berpotensi deadlock (misal kedua process menunggu untuk di-*wakeup*)**

Race Condition
IF Statement

Beberapa process/thread menggunakan shared variable x
Potensi race condition

```
if(x == 0)
{
        x = 1;
}
x++;
```

C code

```
mov  eax, $x
cmp  eax, 0
jne  end
mov  eax, 1
end:
inc eax
mov $x, eax
```

Assembly code

Producer and Consumer (Bounded-buffer problem) dengan sleep and wakeup system call

Race Condition?

Shared variable apa yang tidak dilindungi?

Adakah kemungkinan wakeup message yang hilang?

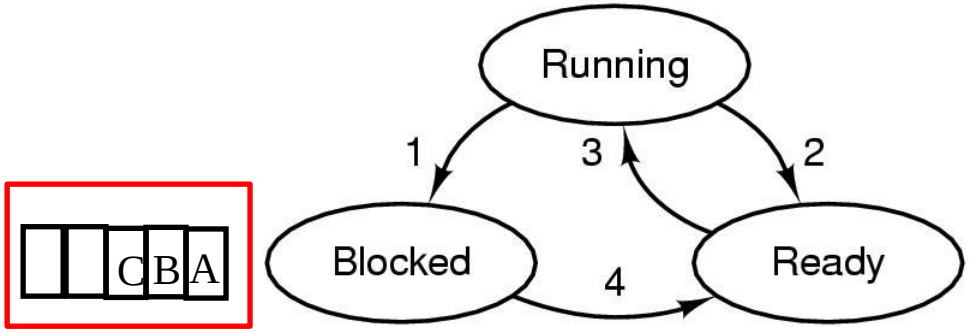
```
#define N 100
int count = 0;

void producer(void)
{
        int item;

        while (TRUE) {
                item = produce_item( );
                if (count == N) sleep( );
                insert_item(item);
                count = count + 1;
                if (count == 1) wakeup(consumer);
        }
}

void consumer(void)
{
        int item;

        while (TRUE) {
                if (count == 0) sleep( );
                item = remove_item( );
                count = count - 1;
                if (count == N - 1) wakeup(producer)
                consume_item(item);
        }
}
```

Process/Thread producer

Process/Thread consumer

Sleep: system call yang menyebabkan proses yang memanggil diblocked (*suspended*)
Wakup: system call yang menyebabkan proses dibangunkan, atau menjadi *ready*

Deadlock



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

- **Diberikan suatu himpunan yang berisikan beberapa process dengan state Blocked. Dikatakan deadlock apabila tiap process di dalam himpunan ini menunggu event yang hanya bisa dimunculkan oleh process lain di himpunan tersebut. Contoh:**
  - **Producer-consumer dengan race condition, producer dan consumer mencapai kondisi sleep dan saling menunggu untuk di-wakeup**

Mekanisme sinkronisasi

Mekanisme sinkronisasi

- **Hardware**
  - **Disable interrupts**
  - **Specific instruction: TSL (Test and Set Lock), XCHG**

- **Software (teknik-teknik pemrograman)**
  - **Condition/lock variable**
  - **Semaphore**
  - **Monitor**
  - **Message passing**
  - **Barriers**

Hardware (1)

- **Disable interrupts**
  - **Process mematikan (disable) semua interrupt setelah memasuki critical region, dan menghidupkan kembali (re-enable) sebelum meninggalkannya**
  - **Sebaiknya tidak dilakukan oleh user process. Kenapa?**
  - **Masih ada problem untuk multiprocessor /multicore system**
    - **Saat interrupts pada satu CPU dimatikan, CPU yang lain masih tetap dapat berjalan dan mengakses shared memory**

Hardware (2)

- **TSL (Test and Set Lock) Instruction →**
  - **Lock memory bus sehingga CPU yang lain tidak dapat mengakses memory**
  - **Berguna untuk multiprocessor system**
    - **Lock memory bus sehingga CPU yang lain tidak dapat mengakses memory**

```
enter_region:
        TSL REGISTER,LOCK          | copy lock to register and set lock to 1
        CMP REGISTER,#0            | was lock zero?
        JNE enter_region           | if it was non zero, lock was set, so loop
        RET | return to caller; critical region entered

leave_region:
        MOVE LOCK,#0               | store a 0 in lock
        RET | return to caller
```

Hardware (3)

- **XCHG Instruction**
  - **Instruksi untuk pertukaran konten dari 2 buah lokasi (misal register dan memory) secara atomic**
  - **Sistem X86 mendukung instruksi ini untuk low level synchronization**

```
enter_region:
        MOVE REGISTER,#1          | put a 1 in the register
        XCHG REGISTER,LOCK        | swap the contents of the register and lock variable
        CMP REGISTER,#0           | was lock zero?
        JNE enter_region          | if it was non zero, lock was set, so loop
        RET                       | return to caller; critical region entered

leave_region:
        MOVE LOCK,#0              | store a 0 in lock
        RET                       | return to caller
```

Semaphores

- **Variable integer khusus yang dikelola oleh OS (diimplementasikan via system calls) dan digunakan untuk menghitung jumlah blocked/sleeps dan unblocked/wakeups**

- **Arti nilai semaphore S:**
  - **Kalau positif → jumlah process yang bisa *decrement* (turunkan satu) S tanpa blocking**
  - **Kalau negatif → jumlah process yang sedang sleep/blocked dan menunggu untuk dibangunkan/wakeup/un-blocked**
  - **Kalau nol → tidak ada process yang menunggu, tapi kalau di-*decrement* (turunkan satu), process ini akan di-blocked**

Operations on Semaphore S

- **Down(S)**

  S = S - 1

  If (S<0) then the calling process is put to sleep/blocked

- **Up(S)**

  S = S + 1

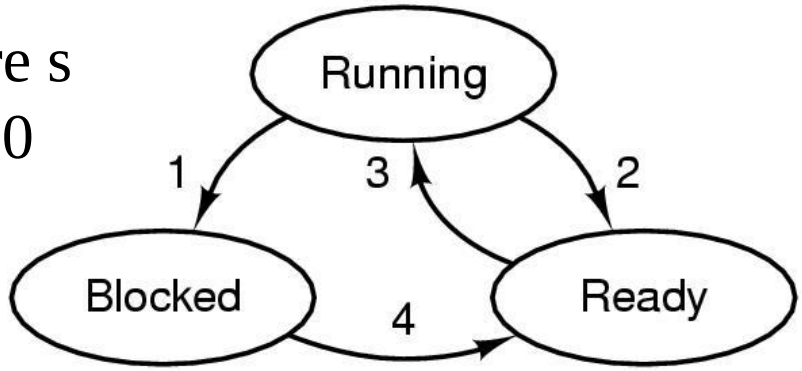
  If (S = 0) then wakeup/unblock the sleep/blocked process

- **Operasi Up() dan Down() bersifat atomic (Atomic Action)**
  - **Artinya saat operasi dimulai, dia tidak dapat diinterupsi sampai operasi selesai, atau sama sekali tidak dijalankan**
  - **Operasi: checking dan updating nilai semaphore, dan action (block atau unblock)**
- **Notasi yang lain menurut karya tulis dari Dijkstra (pengusul konsep semaphore):**

  P() = Down() V() = Up()

Three-State Process Model

Down semaphore s when s = 0



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

Blocked →

Up semaphore s, to change the state to Ready

Arti nilai semaphore:
Kalau positif, artinya nilai tersebut menunjukkan jumlah process yang bisa decrement S tanpa blocking
Kalau negatif, artinya nilai tersebut menunjukkan jumlah process yang sedang sleep/blocked dan menunggu untuk dibangunkan/wakeup/unblocked

Mutex Variable menggunakan Binary Semaphore

- **Mutex variable → 2 states yaitu unlocked dan locked**
  - **Untuk melindungi critical region**

- **Binary semaphore → hanya ada 2 kemungkinan nilai yaitu 0 dan 1**
- **Inisialisasi semaphore ke nilai 1**
- **Agar hanya ada satu process berada di critical region yang dilindungi oleh semaphore, maka lakukan**
  - **Down() untuk masuk critical region**
  - **Up() untuk keluar dari critical region**

- **Contoh:**

```
semaphore mutex = 1;
down(&mutex);
/* Critical
Section */
up(&mutex);
```

Mutex Variable menggunakan Binary Semaphore

Posix Semaphore

Sinkronisasi antar process,
semaphore diletakkan di shared memory

```
sem_init(mutex,1,1)

sem_wait(mutex)

sem_post(mutex)
```

Sinkronisasi antar thread,
semaphore sebagai global variable

```
sem_init(&mutex, 0, 1)

sem_wait(&mutex)

sem_post(&mutex)
```

Contoh: multi-thread & semaphore

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

void thread1();
void thread2();
int g1 = 0;
sem_t mutex;

int main()
{
    int i;
    pthread_t thrd1, thrd2;
    int ret;

    sem_init(&mutex, 0, 1); // inisialisasi binary semaphore
    ret = pthread_create(&thrd1, NULL,(void *)thread1, NULL);
    if (ret)
    {
        perror("pthread_create: thread1");
        exit(EXIT_FAILURE);
    }
    ret = pthread_create(&thrd2, NULL,(void *)thread2, NULL);
    if (ret)
    {
        perror("pthread_create: thread2");
        exit(EXIT_FAILURE);
    }
    pthread_join(thrd2, NULL);
    pthread_join(thrd1, NULL);
    exit(EXIT_SUCCESS);
}
```

```c
void thread1()
{
    while(g1 < 5) {
        sem_wait(&mutex);   // down semaphore
        g1++;
        printf("thread1: g1=%d\n", g1);
        sem_post(&mutex); // up semaphore
        sleep(1);
    }
}

void thread2()
{
    while(g1 < 5) {
        sem_wait(&mutex);   // down semaphore
        g1++;
        printf("thread2: g1=%d\n", g1);
        sem_post(&mutex); // up semaphore
        sleep(1);
    }
}
```

```
ananda@AK-Ubuntu:~/IF535/W7$ gcc -o ex7-sem ex7-sem.c -lpthread
ananda@AK-Ubuntu:~/IF535/W7$ ./ex7-sem
thread2: g1=1
thread1: g1=2
thread2: g1=3
thread1: g1=4
thread2: g1=5
```

Contoh: multi-process & shared memory

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <wait.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <semaphore.h>
#define key ((key_t) 7890)
#define key2 ((key_t) 7891)

int main(int argc, char **argv)
{
    int i,nloop=500000,zero=0,value,*ptr;
    int shmid, semid, mode, shm_size, sem_size,status;
    sem_t *mutex;

   shm_size=sizeof(int);
   /*connect ke segment memory atau create segment memory */
    shmid=shmget(key,shm_size,0644 | IPC_CREAT);

    sem_size=sizeof(sem_t);
    semid=shmget(key2,sem_size,0644 | IPC_CREAT);

   /*attach to the segement to get a pointer to it */
   ptr = (int *)shmat(shmid,(void *)0,0);
    (*ptr) = 0;

    mutex = (sem_t *)shmat(semid,(void *)0,0);
  /* create & initialize semaphore at shared memory */
  if( sem_init(mutex,1,1) < 0)
    {
      perror("semaphore initialization");
      exit(0);
    }
```

Contoh: multi-process & shared memory

```
  if (fork() == 0) { /* child process*/
    for (i = 0; i < nloop; i++) {
        value = ++(*ptr);
        printf("child process %d %d: %d\n", getpid(), getppid(), value); fflush(NULL);
    }
    exit(0);
  }
  /* back to parent process */
  for (i = 0; i < nloop; i++) {
    value = ++(*ptr);
    printf("parent process %d %d: %d\n", getpid(),getppid(),value); fflush(NULL);
  }
    wait(&status);
    shmdt(ptr);
    shmdt(mutex);
    shmctl(shmid,IPC_RMID,(struct shmid_ds *)0);
    shmctl(semid,IPC_RMID,(struct shmid_ds *)0);
    exit(0);
}
```

Berapa jumlah process yang terbentuk?
Variable dan resource apa yang diakses secara bersamaan oleh masing-masing process?
Instruksi apa yang berada di daerah kritis (critical section).
Apa kemungkinan yang dapat terjadi?

Contoh: multi-process & shared memory

```
gcc -o race-shm race-shm.c -lpthread
./race-shm
```

```
child process 5310 5309: 999966
child process 5310 5309: 999967
child process 5310 5309: 999968
child process 5310 5309: 999969
child process 5310 5309: 999970
child process 5310 5309: 999971
child process 5310 5309: 999972
child process 5310 5309: 999973
child process 5310 5309: 999974
child process 5310 5309: 999975
child process 5310 5309: 999976
child process 5310 5309: 999977
child process 5310 5309: 999978
child process 5310 5309: 999979
child process 5310 5309: 999980
child process 5310 5309: 999981
ananda@AK-Ubuntu:~/COURSE-OS-OLD
```

Contoh: multi-process, shared memory, semaphore

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <wait.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <semaphore.h>
#define key ((key_t) 7890)
#define key2 ((key_t) 7891)

int main(int argc, char **argv)
{
    int i,nloop=500000,zero=0,value,*ptr;
    int shmid, semid, mode, shm_size, sem_size,status;
    sem_t *mutex;

   shm_size=sizeof(int);
   /*connect ke segment memory atau create segment memory */
    shmid=shmget(key,shm_size,0644 | IPC_CREAT);

    sem_size=sizeof(sem_t);
    semid=shmget(key2,sem_size,0644 | IPC_CREAT);

   /*attach to the segement to get a pointer to it */
   ptr = (int *)shmat(shmid,(void *)0,0);
    (*ptr) = 0;

    mutex = (sem_t *)shmat(semid,(void *)0,0);
  /* create, initialize semaphore */
  if( sem_init(mutex,1,1) < 0)
    {
      perror("semaphore initialization");
      exit(0);
    }
```

Contoh: multi-process, shared memory, semaphore

```
  if (fork() == 0) { /* child process*/
    for (i = 0; i < nloop; i++) {
        sem_wait(mutex);
        value = ++(*ptr);
        printf("child process %d %d: %d\n", getpid(), getppid(), value); fflush(NULL);
        sem_post(mutex);
    }
    exit(0);
  }
  /* back to parent process */
  for (i = 0; i < nloop; i++) {
    sem_wait(mutex);
    value = ++(*ptr);
    printf("parent process %d %d: %d\n", getpid(),getppid(),value); fflush(NULL);
    sem_post(mutex);
  }
    wait(&status);
    shmdt(ptr);
    shmdt(mutex);
    shmctl(shmid,IPC_RMID,(struct shmid_ds *)0);
    shmctl(semid,IPC_RMID,(struct shmid_ds *)0);
    exit(0);
}
```

Contoh: multi-process, shared memory, semaphore

```
gcc -o sem-shm sem-shm.c -lpthread
```

```
./sem-shm
```

```
child process 5392 5391: 999984
child process 5392 5391: 999985
child process 5392 5391: 999986
child process 5392 5391: 999987
child process 5392 5391: 999988
child process 5392 5391: 999989
child process 5392 5391: 999990
child process 5392 5391: 999991
child process 5392 5391: 999992
child process 5392 5391: 999993
child process 5392 5391: 999994
child process 5392 5391: 999995
child process 5392 5391: 999996
child process 5392 5391: 999997
child process 5392 5391: 999998
child process 5392 5391: 999999
child process 5392 5391: 1000000
ananda@AK-Ubuntu:~/COURSE-OS-OLD
```

Producer-Consumer

Producer-Consumer Problem menggunakan semaphores

```
sem_init(&empty, 0, SIZE);
sem_init(&full, 0, 0);
sem_init(&mutex, 0, 1);
```

Inisialisasi semaphore untuk multi-thread application

```
#define N 100
typedef int semaphore;
semaphore mutex = 1;
semaphore empty = N;
semaphore full = 0;

void producer(void)
{
        int item;

        while (TRUE) {
                item = produce_item( );
                down(&empty);
                down(&mutex);
                insert_item(item);
                up(&mutex);
                up(&full);
        }
}


void consumer(void)
{
        int item;

        while (TRUE) {
                down(&full);
                down(&mutex);
                item = remove_item( );
                up(&mutex);
                up(&empty);
                consume_item(item);
        }
}
```

Event semaphores
Hitung jumlah slot kosong
Hitung jumlah slot yang terisi

- **Apa yang terjadi kalau terjadi kesalahan urutan penggunaan semaphore? Misal:**

```
void producer()
        ...
        down(&mutex);
        down(&empty);
        ...
        up(&mutex);
        up(&full);
        ...
```

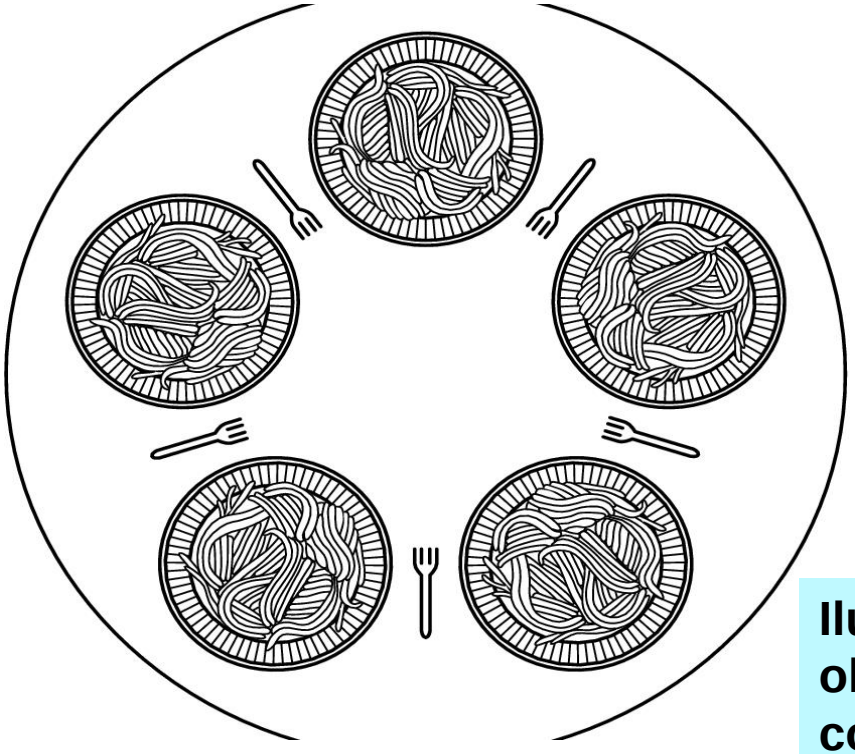
Contoh:

```
void producer(void)
{
        int item;

        while (TRUE) {
                item = produce_item( );
                down(&empty);
                down(&mutex);
                insert_item(item);
                up(&mutex);
                up(&full);
        }
}
```

```
sem_wait(&empty);
sem_wait(&mutex);
sem_getvalue(&empty, &item);
prod = (SIZE-1) - item;
data[prod] = prod;
printf("Producer: %d %d \n",prod, data[prod]);
sem_post(&mutex);
sem_post(&full);
```

Dining Philosophers

Dining Philosophers



Contoh 5 philosophers dan 5 forks

Ilustrasi koordinasi akses shared resources oleh process/thread yang berjalan secara concurrent

Terdapat N philosophers yang terus menerus Think (berpikir) dan Eat (makan)
Tiap kali lapar, ambil fork (garpu) kiri lalu kanan (atau urutan sebaliknya).
Dengan 2 fork (kiri dan kanan), philosopher dapat makan spaghetti
Dua philosophers tidak bisa menggunakan fork yang sama pada waktu yang sama (mutual exclusion), dan tidak terjadi kasus deadlock dan starvation

Dining Philosophers: Contoh solusi yang bermasalah

```
#define N 5                                     /* number of philosophers */

void philosopher(int i)                         /* i: philosopher number, from 0 to 4 */
{
    while (TRUE) {
        think( );                               /* philosopher is thinking */
        take_fork(i);                           /* take left fork */
        take_fork((i+1) % N);                   /* take right fork; % is modulo operator */
        eat( );                                 /* yum-yum, spaghetti */
        put_fork(i);                            /* put left fork back on the table */
        put_fork((i+1) % N);                    /* put right fork back on the table */
    }
}
```

5 process/thread menjalankan procedure philosopher()
Apa masalah dari code di atas?

Bagaimana kalau dimodifikasi sehingga setelah ambil left fork, periksa apakah right fork tersedia. Kalau right fork tidak tersedia, taruh left fork, tunggu beberapa lama, dan ulangi lagi pengambilan fork

Bagian mana yang di-shared dan harus dilindungi?
Bagaimana kalau ke-5 statement setelah think dilindungi dengan mutex?

Dining Philosophers: Solusi dengan semaphores (1)

```
#define N                5                  /* number of philosophers */
#define LEFT             (i+N–1)%N          /* number of i's left neighbor */
#define RIGHT            (i+1)%N            /* number of i's right neighbor */
#define THINKING         0                  /* philosopher is thinking */
#define HUNGRY           1                  /* philosopher is trying to get forks */
#define EATING           2                  /* philosopher is eating */
typedef int semaphore;                      /* semaphores are a special kind of int */
int state[N];                               /* array to keep track of everyone's state */
semaphore mutex = 1;                        /* mutual exclusion for critical regions */
semaphore s[N];                             /* one semaphore per philosopher */

void philosopher(int i)                     /* i: philosopher number, from 0 to N–1 */
{
    while (TRUE) {                          /* repeat forever */
        think( );                           /* philosopher is thinking */
        take_forks(i);                      /* acquire two forks or block */
        eat( );                             /* yum-yum, spaghetti */
        put_forks(i);                       /* put both forks back on table */
    }
}
```

Dining Philosophers: Solusi dengan semaphores (2)

```
. . .

void take_forks(int i)                  /* i: philosopher number, from 0 to N–1 */
{
        down(&mutex);                   /* enter critical region */
        state[i] = HUNGRY;              /* record fact that philosopher i is hungry */
        test(i);                        /* try to acquire 2 forks */
        up(&mutex);                     /* exit critical region */
        down(&s[i]);                    /* block if forks were not acquired */
}

. . .
```

Dining Philosophers: Solusi dengan semaphores (3)

. . .

```
void put_forks(i)                          /* i: philosopher number, from 0 to N–1 */
{
        down(&mutex);                      /* enter critical region */
        state[i] = THINKING;               /* philosopher has finished eating */
        test(LEFT);                        /* see if left neighbor can now eat */
        test(RIGHT);                       /* see if right neighbor can now eat */
        up(&mutex);                        /* exit critical region */
}

void test(i) /* i: philosopher number, from 0 to N–1 */
{
        if (state[i] == HUNGRY && state[LEFT] != EATING && state[RIGHT] != EATING) {
                state[i] = EATING;
                up(&s[i]);
        }
}
```

Mekanisme Sinkronisasi Lainnya:
- Monitor
-Message Passing

Monitors

- **Higher level synchronization (language concept)**
- **Kumpulan procedure, variable, dan struktur data yang dikelompokkan menjadi satu modul atau package**
- **Process dapat memanggil procedures yang ada di dalam monitor, tapi tidak dapat mengakses struktur data dari monitor**

- **Hanya ada satu process yang aktif di dalam monitor pada satu waktu**
  - **Compiler membuat call ke monitor procedures berbeda dengan procedures biasa → ada beberapa instruksi ditambahkan untuk memeriksa apakah ada process lain yang sedang aktif di dalam monitor**
  - **Pada Java, penggunaan monitor dengan synchronized method/statement**

Producer-consumer problem menggunakan monitor

```
procedure producer;
begin
    while true do
    begin
        item = produce_item;
        ProducerConsumer.insert(item)
    end
end;
procedure consumer;
begin
    while true do
    begin
        item = ProducerConsumer.remove;
        consume_item(item)
    end
end;
```

```
monitor ProducerConsumer
    condition full, empty;
    integer count;
    procedure insert(item: integer);
    begin
        if count = N then wait(full);
        insert_item(item);
        count := count + 1;
        if count = 1 then signal(empty)
    end;
    function remove: integer;
    begin
        if count = 0 then wait(empty);
        remove = remove_item;
        count := count - 1;
        if count = N - 1 then signal(full)
    end;
    count := 0;
end monitor;
```

Contoh implementasi monitor dalam Java

```
public class ProducerConsumer {
        static final int N = 100;       // constant giving the buffer size
        static producer p = new producer( );    // instantiate a new producer thread
        static consumer c = new consumer( ); // instantiate a new consumer thread
        static our_monitor mon = new our_monitor( );      // instantiate a new monitor

        public static void main(String args[ ]) {
           p.start( );      // start the producer thread
           c.start( );      // start the consumer thread
        }

        static class producer extends Thread {
           public void run( )  { // run method contains the thread code
              int item;
              while (true) {     // producer loop
                 item = produce_item( );
                 mon.insert(item);
              }
           }
           private int produce_item( ) { ... }      // actually produce
        }

        static class consumer extends Thread {
           public void run( )  { run method contains the thread code
              int item;
              while (true) {     // consumer loop
                 item = mon.remove( );
                 consume_item (item);
              }
           }
           private void consume_item(int item) { ... } // actually consume
        }
```

Contoh implementasi monitor dalam Java

```
static class our_monitor {   // this is a monitor
    private int buffer[ ] = new int[N];
    private int count = 0, lo = 0, hi = 0;   // counters and indices

    public synchronized void insert(int val) {
        if (count == N) go_to_sleep( );     // if the buffer is full, go to sleep
        buffer [hi] = val; // insert an item into the buffer
        hi = (hi + 1) % N;        // slot to place next item in
        count = count + 1;        // one more item in the buffer now
        if (count == 1) notify( );        // if consumer was sleeping, wake it up
    }

    public synchronized int remove( ) {
        int val;
        if (count == 0) go_to_sleep( );     // if the buffer is empty, go to sleep
        val = buffer [lo]; // fetch an item from the buffer
        lo = (lo + 1) % N;        // slot to fetch next item from
        count = count – 1;        // one few items in the buffer
        if (count == N – 1) notify( ); // if producer was sleeping, wake it up
        return val;
    }
    private void go_to_sleep( ) { try{wait( );} catch(InterruptedException exc) {};}
}
}
```

Message Passing (1)

- **Kernel menyediakan 2 primitives:**
  - **Send (destinationm&message)**
  - **Receive (source, &message)**

- **Kelebihan**
  - **Sinkronisasi sebagai bagian dari primitives yang disediakan oleh kernel**
  - **Komunikasi antar process pada suatu jaringan komputer**
    - Portabilitas untuk aplikasi-aplikasi terdistribusi (distributed systems)
- **Kekurangan:**
  - **Tidak secepat shared memory untuk pertukaran pesan antar 2 processes di komputer yang sama**
  - **Perlu protocol untuk antisipasi masalah-masalah pada jalur komunikasi, mis. Message lost, acknowledgement, clock synchronization, dsb.**

Message Passing (2)

- **Implementasi**
  - **Komunikasi**
    - Direct: komunikasi langsung antar processes sbg sender dan receiver
    - Indirect: komunikasi tidak langsung via mailbox (contoh: message queue pada Posix IPC)
  - **Naming/Addressing**
    - Host id, process id, port number, authentication
  - **Synchronous atau asynchronous**
    - Synchronous: blocked sender menunggu message diterima oleh receiver dan blocked receiver sampai ada message yang dikirim
    - Asynchronous: melanjutkan instruksi yang lain tanpa menunggu message diterima atau dikirim

Producer-Consumer menggunakan message passing

```
#define N 100

void producer(void)
{
        int item;
        message m;

        while (TRUE) {
                item = produce_item( );
                receive(consumer, &m);
                build_message(&m, item);
                send(consumer, &m);
        }
}

void consumer(void)
{
        int item, i;
        message m;

        for (i = 0; i < N; i++) send(producer, &m)
        while (TRUE) {
                receive(producer, &m);
                item = extract_item(&m);
                send(producer, &m);
                consume_item(item);
        }
}
```

Akhir Kuliah Minggu 5

Terima kasih atas perhatiannya!